

EECS 498 · AASE · FALL 2026

Course intro + the AI coding landscape

LECTURE 01 · SEP 1, 2026

Smile.

Smile. You belong here.

Where we're headed today

- **One object, three verbs** — what we'll do this semester
- **How this course runs** — where things live, grading, what it asks
- **AI coding landscape** — wave history, where we are in 2026
- **Code curator philosophy** — the mental model
- **Discussion** — two questions
- **The tool** — Aider, and what it feels like
- **This week** — `aider-practice` + setup gate + what you're building

Course Shape

APPLY · ANALYZE · CREATE

One object. Three verbs.



Apply • Wks 1–3

Use a coding agent to build one.
Aider against your own endpoint
→ a pair-programmer.



Analyze • Wks 4–7

Take yourself out of the loop.
Tools, stop conditions, then
measure what broke.



Create • Wks 8–15

Grow it into an assistant. Memory,
method, and a way to reach it that
isn't a terminal.

How This Course Runs

[SITES](#) · [STAFF](#) · [GRADES](#) · [CALENDAR](#)

Where things live

Course site <code>eeecs498-aase.github.io</code>	Syllabus + schedule. Wins on dates and policy
Ed <code>edstem.org</code>	Questions and announcements . This is where we reach you
GitHub <code>aase-student</code>	Everything you receive. Your own repo per handout. Accept the invite in your email
Canvas	Grades
Podcast (optional)	A Brief before each lecture, a Deep Dive after

Lectures are recorded (CAEN + Zoom). Podcast episodes are AI-generated from course materials and *de-identified* transcripts. Nobody is named. Want something out? Email the staff address; it comes out.

Who you're dealing with

- **Marcus Darden** — instructor
- **Rafe Symonds, Zilin Wang** — GSIs
- **Rushil Kagithala** — IA

`eeecs-aase-staff@umich.edu` reaches all four.

Personal things (extensions, accommodations, hardware) → email. Anything the class would benefit from hearing → Ed.

How you're graded

	Share of your grade
Administrative — lecture + lab attendance, bonuses	10%
Apply — weeks 1–3, build to end of 4	18%
Analyze — weeks 4–7	22.5%
Create — weeks 8–15	49.5%

The three phases split the other 90% as **20 / 25 / 55**.

28 lectures • 13 labs • 3 evenings



Lectures

Tue + Thu, 3:00–4:30, LCSIB 1355



Labs

Mon 3:30 (GGBL 2153) or Tue 12:30 (DOW 1005). Same content, attend either



Hackathons

Evenings, whole cohort, weeks 4 • 7 • 15. First: **Thu Sep 24**

Put Sep 24 in your calendar tonight. If an evening is genuinely impossible for you, tell us early and we build a makeup path.

The short version

- **Attendance** — **10% of your grade**, 60% lecture / 40% lab. Poll Everywhere, app or browser. **70% earns full credit**
- **Late work** — 10%/day for 3 days, then zero. No late days
- **Individual work** — talk about ideas freely, never share code or prompts
- **Costs** — *nothing*. No textbook, no LLM spend. If your laptop can't serve a model, CAEN covers you. Details on Ed

If you can **explain** it, it's yours.

Where We Are

FALL 2026

A brief history

- **Wave 1 — Autocomplete** (2021–22). GitHub Copilot. One file, you drive.
- **Wave 2 — Chat** (2022–23). ChatGPT, then chat inside the IDE. Ask, paste, adapt.
- **Wave 3 — Agentic tools** (2023–25). Cursor and Aider, then Claude Code and Codex. Multi-file edits; the tool runs commands.
- **Wave 4 — Orchestration** (2025–now). Several agents, long-running, supervised. Where Create takes us.

The hype was steep. The reality is
more interesting.

Take Five

TALK TO YOUR NEIGHBOR

Compare notes

Think about the last time you wrote code with help from an AI tool — or the last time you wrote code *without* one. What felt productive? What felt frustrating? When you finished, did you feel like you had *learned* something, or just *gotten an answer*?

This question sits underneath everything in this course.

Code Curator

THE PHILOSOPHY

Typist



Writes lines

Syntax, formatting, boilerplate.



Spends time on HOW

Implementation mechanics.



Owns the keystrokes

Correct means "I typed it right."

Curator



Specifies intent

What and why, not how.



Evaluates output

Reviews, redirects, accepts.



Owns the outcome

Correct means "it works and I can defend it" — whoever typed it.

KISS: Keep prompts focused; resist the urge to over-explain.

PollEv

Bad

Writing something new

Please could you help me write a Python function that iterates through a list of integers, checks if each one is even, and if it is even adds it to a new list... actually I think a list comprehension is cleaner but I'm not sure...

Chasing a failure

The tests are failing. Something with dates? Or the timezone maybe? Can you look at it and just make it work.

Good

Writing something new

Write a function that extracts the even numbers from a list.

Chasing a failure

test_parse_date fails on "2026-02-30". Make parse_date raise ValueError on invalid dates instead of returning None.

The Big Three



Context

What does the AI know when it starts?



Model

Which AI? Capability vs cost vs speed.



Prompt

How do you specify what you want?

L03 goes deep. Today's preview only.

Discussion

TWO QUESTIONS

TAKE ONE MINUTE EACH • THEN WE DISCUSS

Two questions

1. If AI can write code, why do you need EECS 281?
2. When would you *not* want to use AI to write code?

The Tool

WHY AIDER — FOR NOW

Why Aider in Apply?

- **Big Three are explicit.** No GUI hides context, model, or prompt decisions.
- **Speaks the OpenAI-compatible API.** So it talks to the model *you* serve.
- **Small enough to rebuild.** And you will, starting week 3.

AI Pair-Coding

A FIRST LOOK

What the loop feels like

Prompt 1 → "Refactor parse_args() to handle no-args case cleanly; print usage and exit."

Prompt 2 → "Add a unit test for the no-args path."

Prompt 3 → "Replace the if-chain in dispatch() with a dictionary lookup."

Three coordinated changes. Tests pass. I wrote zero lines of code.

I described outcomes. The model
produced them. I **judged**.

This Week

AIDER-PRACTICE

•

SETUP GATE

•

THE BUILD

Every lesson is a failing test

- **Part 1, lessons 1–10** this week — build `taskr`’s command line on `qwen3.5:4b`
- **Part 2, lessons 11–16** next week — a web front end on the same data. `9b` *recommended*, not required
- You drive Aider until the test passes. *You never have to wonder whether you’re done*
- **Setup gate + all sixteen lessons due Fri Sep 11**
- **Lab01** Mon Sep 14 / Tue Sep 15 — the pivot lab

You already got Aider running in the Setup Lab. Stuck after that? Office hours — any of us, not just me. Times and rooms on Ed by Friday.

Your first project



What you build

A **pair-programmer**. Everyone builds the same thing: **seven fixed features** on a starter we give you



When

Build it in **weeks 3–4**, about **6 hrs/week**. Planned due **Fri Sep 25**



What you hand in

Code that passes the **feature gate**, plus your spec, session logs, and a short reflection

This is the shape, not the spec. The spec — or its release date — comes at **L03, Tue Sep 8**. Nothing to do about it this week: you are in `aider-practice`.

Aider will be gone. The **principles** won't.

Questions?